RACER-GT Python API Reference

Public Workflow and Diagnostic Entry Points

Yi-Hao Lai

Department of Finance, Da-Yeh University

2026-07-27 • version 1.0.0

Abstract

The top-level API is stable. Lower-level functions remain available for methodological diagnostics but may evolve faster than the pipeline interface. Where a routine raises rather than returning a degraded result, this document says so, because those refusals are deliberate design decisions rather than missing error handling.

Contents

1 Top-level API

```
from racergt import (
    RacerGTConfig, RacerGTPipeline, PipelineResult,
    DesignWeightedResult, fit_design_weighted_consensus,
    generate_collection_schedule, generate_chunk_windows,
)
```

1.1 `RacerGTConfig`

A nested pydantic model holding every locked setting. All sub-models use `extra="forbid"`, so an unrecognised YAML key is a validation error rather than a silently ignored typo.

```
cfg = RacerGTConfig.load_yaml("protocol.yaml")
cfg.save_yaml("protocol.lock.yaml")
digest = cfg.protocol_hash()          # SHA-256 of the canonical
   serialization
```

`load_yaml` recomputes the hash and raises if a stored `protocol_hash` disagrees. Hand-editing a lock file therefore fails loudly. To change a setting, edit the source configuration and regenerate.

1.2 `generate_chunk_windows(start, end, window_days, step_days)`

Returns the fixed overlapping chunk manifest: `chunk_id`, `window_start`, `window_end`, `n_calendar_days`.

1.3 `generate_collection_schedule(config, anchor_date=None)`

Returns the balanced day × stream × replicate schedule with stream order, planned time slot, deterministic chunk order, fixed historical endpoints, and the protocol hash.

Remark 1.1. `collection_day` *in the output is the* ordinal design level *(0, 1, 2, ...), not the day offset;* `day_offset` *and* `planned_collection_date` *are separate columns.*

1.4 `RacerGTPipeline(config).fit(raw, benchmark=None)`

Runs audit, per-pull overlap calibration, duplicate diagnostics, the design-based reference estimator, G-studies, covariance-adjusted consensus, temporal benchmarking when a benchmark is supplied, reliability diagnostics, and the locked decision tree. Returns a `PipelineResult`.

Raises `ValueError` when the raw batch fails the protocol audit, unless `stop_on_audit_er`

1.5 `PipelineResult`

Attribute	Contents
`final_series`	final daily index, conditional SE, 95% interval
`calibrations`	per-pull overlap-graph calibration results
`duplicate_diagnostics`	exact groups and the near-duplicate graph
`gstudies`	`level`, `detection`, `innovation`
`design_consensus`	design-based reference estimator
`consensus`	GLS / minimum-variance consensus
`benchmark`	temporal benchmark result, or `None`
`reliability`	pairwise, day/stream, convergence diagnostics
`decision`	PASS / REVIEW / FAIL and every rule
`audit`	protocol-integrity audit

`final_series` returns the benchmark result when one exists and the GLS consensus otherwise. `design_consensus` never feeds it.

2 Diagnostic entry points

2.1 Calibration

```
from racergt.overlap import OverlapGraphCalibrator, huber_location,
   OverlapGraphError
```

`OverlapGraphCalibrator.fit` expects exactly one `series_id` and one `pull_id`. It raises `OverlapGraphError` when the overlap graph is disconnected and `allow_disconnect` is false, because a disconnected graph means the relative scales are only partially identified and returning a series anyway would conceal that.

2.2 Duplicates, G-study, consensus

```
from racergt.duplicates import diagnose_duplicates
from racergt.gstudy import run_gstudy, run_all_gstudies
from racergt.consensus import fit_design_weighted_consensus,
   fit_gls_consensus
```

`run_gstudy` raises if the design is not fully crossed or replicate counts are unequal, rather than estimating components from an unbalanced design. `fit_gls_consensus` requires at least two pulls and a positive finite baseline mean for each.

Remark 2.1 (Duplicate components are not equivalence classes). *Connected components of the near-duplicate graph are connectivity sets. Membership does not imply every pair inside exceeds the threshold, because connectivity is transitive and the pairwise rule is not.*

2.3 Benchmark and measurement error

```
from racergt.benchmark import temporal_benchmark
from racergt.eiv import reliability_corrected_ols, simex_ols
```

`reliability_corrected_ols` raises when the corrected denominator is nonpositive. That condition means the estimated measurement error accounts for all observed regressor variation; a number returned there would be meaningless.

2.4 Simulation and validation

```
from racergt.simulation import SimulationSettings, simulate_racergt_data
from racergt.validation import run_monte_carlo
```

3 File I/O

`racergt.io.read_table` reads CSV, Parquet, XLS/XLSX; `write_table` writes CSV, Parquet, XLSX, and Stata 118 `.dta`.

4 Command line

Command	Purpose	Exit
`racergt design`	lock protocol, schedule, chunk manifest	0
`racergt audit`	check a batch against the locked protocol	2 on failure
`racergt run`	run the full pipeline	3 on FAIL
`racergt simulate`	controlled dataset, optionally run pipeline	0
`racergt export-stata`	CSV to Stata 118	0